

Functional Architecture Blueprint

LinkBit - Functional Architecture

1. Core Security & Trust Model

Trust Philosophy

“**Minimized Trust**” using **Bitcoin-native enforcement + protocol-defined rules**, not human discretion.

Escrow Model

- **2-of-3 Multisignature Bitcoin Escrow**
- **Non-custodial by design:** No single party can move funds alone.

Key Holders

- **Borrower:** Key 1 (Client-side, non-custodial)
- **Lender:** Key 2 (Client-side, non-custodial)
- **Platform:** Key 3 (Server-side, secured via AWS KMS / HSM)

Platform Role

- Acts as a **neutral protocol enforcer and mediator**
- Executes **pre-agreed rules only**
- Cannot:
 - Move funds unilaterally
 - Change loan terms post-signing
 - Liquidate without protocol conditions being met

The platform enforces *rules*, not subjective fairness.

2. Phase 1 – Discovery & Connection

Goal

Privately match borrowers and lenders while preserving anonymity.

Loan Offer Creation (Lender-Driven)

Each lender posts a **single, self-contained loan offer**:

- Loan Amount (INR)
- Interest Rate (fixed or bounded range)
- Maximum LTV
- Tenure options
- Liquidation thresholds

Constraint (MVP):

- **1 Offer = 1 Loan Agreement**
- No partial fills (simplifies escrow & risk)

Marketplace Browsing (Borrower)

- Borrowers view an **anonymized feed**
- Filters:
 - Amount
 - Tenure
 - Interest rate
- Real identities hidden (pseudonyms only)

Connection Flow

1. Borrower selects an offer

2. System validates:
 - KYC status
 - Basic eligibility
 3. A **private negotiation channel** is opened
-

3. Phase 2 – Negotiation & Contract Formation

Goal

Convert negotiation into a **legally binding + technically enforceable contract**.

Real-Time Negotiation

- Secure chat between borrower and lender
- Live term sheet side panel
- **Only lender can modify financial terms**

Handshake & Freeze

1. Verbal agreement reached
2. Lender clicks **“Finalize Offer”**
3. System actions:
 - Locks chat (read-only)
 - Freezes term sheet (immutable)
 - Generates a **Digital Loan Agreement**

Agreement Contents

- Final loan terms
- Margin call thresholds

- Liquidation thresholds
- Grace periods
- Reference BTC price used
- Fiat repayment schedule

Digital Signatures

- Agreement is hashed (SHA-256)
- Lender signs hash
- Borrower signs hash
- Hash + signatures stored immutably (audit log)

Status → **AWAITING_PLATFORM_FEE**

4. Platform Processing Fee (Gatekeeper)

Purpose

- Prevent spam
- Cover escrow & monitoring costs
- Allocate expensive resources only to serious users

Flow

1. Borrower proceeds
2. Platform requests **fixed processing fee** (UPI / Netbanking)
3. Payment gateway confirms success
4. Fee is **refundable if escrow is not generated within SLA**

Status → **ESCROW_INITIATION**

5. Phase 3 – Collateral & Escrow Setup

Goal

Lock Bitcoin collateral on-chain.

Escrow Generation

- Platform retrieves public keys:
 - Borrower
 - Lender
 - Platform
- Generates **2-of-3 P2WSH SegWit address**
- Address shown to borrower

Collateral Deposit

- Borrower sends BTC to escrow address
- Platform monitors blockchain

Confirmation Logic

- 0 conf → **DEPOSIT_DETECTED**
- ≥ 1 confirmation → **COLLATERAL_LOCKED**

Confirmation count is configurable based on loan size.

Lender Notification

“Collateral secured. Safe to disburse funds.”

6. Phase 4 – Fiat Disbursement

Goal

Bridge fiat rails without compromising Bitcoin security.

Identity Reveal

- Borrower's real bank/UPI details revealed **only after collateral lock**

Disbursement

- Lender transfers INR via IMPS / NEFT / UPI
- Lender marks loan as **"Disbursed"**
- Uploads transaction reference

Verification

- Primary: Open Banking / UPI APIs
- Fallback: Borrower confirms receipt

Dispute Resolution (Protocol-Defined)

If verification fails within **X hours**:

- **Default outcome** → Collateral refunded to borrower
- Reason: Fiat rails are lender-controlled

Manual admin intervention:

- Only for resolving contradictions
- Admin signs **only according to protocol outcome**

Status → **ACTIVE**

7. Phase 5 – Active Loan Management

EMI Repayments

- Borrower repays via fiat
- Auto-confirmed via APIs or lender confirmation
- System ledger updates:
 - Outstanding principal
 - Interest accrued

Price Oracle System (Watchtower)

- Aggregates BTC prices from **multiple exchanges**
- Uses **median price** to avoid manipulation
- Polling interval: configurable (e.g., 1 minute)

Failure Handling

- If one oracle fails → ignored
- If multiple fail → liquidation paused

LTV Monitoring

$$LTV = \frac{\text{Outstanding Loan (INR)}}{\text{BTC Collateral Value (INR)}}$$

$$LTV = \frac{\text{BTC Collateral Value (INR)}}{\text{Outstanding Loan (INR)}}$$

Thresholds

- **Warning LTV** → Notification
- **Margin Call LTV** → Borrower must act within X hours
- **Liquidation LTV** → Eligible for liquidation

Borrower can:

- Add BTC
 - Repay fiat
 - Do nothing
-

8. Phase 6 – Closure & Settlement

Scenario A: Normal Closure

- Loan fully repaid
 - Platform generates release transaction
 - Signatures:
 - Platform
 - Lender
 - BTC returned 100% to borrower
 - Status → **CLOSED**
-

Scenario B: Liquidation

Trigger Conditions

- $LTV \geq$ liquidation threshold **OR**
- Repayment overdue beyond grace period

Cooling-Off Period

- Liquidation eligibility does **not** mean immediate execution
 - Borrower gets final window to act
-

Fair Liquidation Calculation

Variables

- DDD: Total debt (principal + interest + penalties)
- PPP: Median BTC price
- BBB: Volatility buffer (e.g., 3%)
- CCC: Total BTC collateral

Lender BTC Share

$$L_{\text{btc}} = \min\left(C, \frac{D}{P} \times (1 + B)\right)$$

Borrower BTC Share

$$R_{\text{btc}} = C - L_{\text{btc}}$$

If collateral is insufficient, lender receives 100% of collateral.

Execution

1. Platform generates PSBT with:
 - Output 1 → Lender
 - Output 2 → Borrower (if any)
2. Platform signs
3. Lender signs
4. Transaction broadcast (platform or lender)
5. Status → **LIQUIDATED**

Lender independently sells BTC off-platform.

9. Explicit Non-Goals & Assumptions

- Platform does **not** custody funds
- Platform does **not** guarantee fiat transfers
- Platform does **not** protect against BTC volatility
- Platform enforces **rules, not discretionary fairness**
- All actions are governed by **pre-signed agreements**

Master Database Schema Specification

LinkBit - Database Schema Specification

1. User & Identity Management

Table: **users**

Purpose: Core authentication and public-facing identity.

Column	Type	Description
id	UUID (PK)	Unique user identifier
email	VARCHAR (UNIQUE)	Login email
password_hash	VARCHAR	Argon2/Bcrypt hash
pseudonym	VARCHAR	Marketplace display name
user_type	ENUM	BORROWER, LENDER, BOTH
kyc_status	ENUM	PENDING, VERIFIED, REJECTED
created_at	TIMESTAMP	Account creation time
updated_at	TIMESTAMP	Last profile update

Table: **user_kyc_details**

Purpose: Sensitive identity data (restricted access, app-level encrypted).

Column	Type	Description
user_id	UUID (PK, FK → users.id)	User reference
full_legal_name	VARCHAR	Legal name
phone_number	VARCHAR	Verified mobile

bank_account_details	JSONB	Encrypted bank/UPI data
national_id_hash	VARCHAR	Hash of PAN/Aadhaar
created_at	TIMESTAMP	KYC submission time

2. Marketplace & Discovery

Table: **loan_offers**

Purpose: Lender-posted loan templates (MVP: 1 offer → 1 loan).

Column	Type	Description
id	UUID (PK)	Offer ID
lender_id	UUID (FK → users.id)	Offer owner
max_loan_amount_in_r	DECIMAL	Maximum loan size
min_interest_rate	DECIMAL	Minimum APY
max_interest_rate	DECIMAL	Maximum APY
max_ltv_percent	INT	Maximum LTV allowed
status	ENUM	OPEN, PAUSED, CLOSED
created_at	TIMESTAMP	Offer creation
updated_at	TIMESTAMP	Last update

3. Loan Lifecycle (Core State Machine)

Table: **loans**

Purpose: Central record of each loan agreement.

Column	Type	Description
--------	------	-------------

id	UUID (PK)	Loan ID
offer_id	UUID (FK → loan_offers.id)	Source offer
borrower_id	UUID (FK → users.id)	Borrower
lender_id	UUID (FK → users.id)	Lender
status	ENUM	See Loan Status Enum
principal_amount	DECIMAL	Agreed loan amount
interest_rate_apy	DECIMAL	Final APY
duration_days	INT	Loan tenure
collateral_btc_amount	DECIMAL	BTC locked
agreement_hash	VARCHAR	SHA-256 hash
lender_signature	TEXT	Signature of hash
borrower_signature	TEXT	Signature of hash
fiat_disbursed_at	TIMESTAMP	Lender claims disbursement
fiat_received_confirmed_at	TIMESTAMP	Borrower confirmation
liquidation_price_used	DECIMAL	BTC price at liquidation
liquidation_ltv	DECIMAL	LTV at liquidation
liquidation_executed_at	TIMESTAMP	Liquidation time
created_at	TIMESTAMP	Loan creation
updated_at	TIMESTAMP	State update time

Loan Status Enum

NEGOTIATING: Chat is open, terms are fluid.

AWAITING_FEE: Contract signed, waiting for Platform Fee payment.

AWAITING_COLLATERAL: Fee paid, waiting for Bitcoin deposit.

COLLATERAL_LOCKED: Bitcoin confirmed, waiting for Fiat disbursement.

ACTIVE: Money disbursed, loan is live.

MARGIN_CALL: LTV breached warning threshold.

LIQUIDATION_ELIGIBLE: LTV breached critical limit or Default.

SETTLEMENT_PENDING: Loan paid, waiting for collateral release signature.

CLOSED: Completed successfully.

LIQUIDATED: Collateral sold to cover debt.

DISPUTE_OPEN: Manual intervention required.

4. Platform Revenue

Table: **platform_fees**

Purpose: Track processing fees collected before escrow creation.

Column	Type	Description
id	UUID (PK)	Fee transaction ID
loan_id	UUID (FK → loans.id)	Loan reference
amount_inr	DECIMAL	Fee amount
payment_gateway_ref	VARCHAR	Gateway reference
status	ENUM	PENDING, SUCCESS, FAILED
created_at	TIMESTAMP	Fee attempt time

5. Escrow & Blockchain Data

Table: **escrow_accounts**

Purpose: Store multisig escrow metadata (no private keys).

Column	Type	Description
loan_id	UUID (PK, FK → loans.id)	1:1 with loan
borrower_pubkey	VARCHAR	Borrower pubkey
lender_pubkey	VARCHAR	Lender pubkey
platform_key_id	VARCHAR	AWS KMS key alias
redeem_script_hex	TEXT	Multisig redeem script
address	VARCHAR	P2WSH address
current_balance_sats	BIGINT	Cached balance (non-authoritative)

Table: bitcoin_transactions

Purpose: Immutable audit log of all on-chain activity.

Column	Type	Description
tx_hash	VARCHAR (PK)	Bitcoin TXID
loan_id	UUID (FK → loans.id)	Loan reference
type	ENUM	DEPOSIT, REFUND, LIQUIDATION, RELEASE
amount_sats	BIGINT	BTC amount
from_addresses	VARCHAR	Source address
to_address	VARCHAR	Destination address
confirmations	INT	Block confirmations
raw_hex	TEXT	Raw transaction
created_at	TIMESTAMP	Detection time

6. Fiat Payments & Risk Monitoring

Table: **repayments**

Purpose: Track EMI payments and verification.

Column	Type	Description
id	UUID (PK)	Repayment ID
loan_id	UUID (FK → loans.id)	Loan reference
due_date	DATE	Scheduled date
amount_due	DECIMAL	Expected EMI
amount_paid	DECIMAL	Paid amount
payment_proof_url	VARCHAR	Receipt/Screenshot
status	ENUM	DUE, PAID_UNVERIFIED, VERIFIED, OVERDUE
verified_by	ENUM	AUTO_API, LENDER_MANUAL, ADMIN_OVERRIDE
created_at	TIMESTAMP	Record time

Table: **ltv_logs**

Purpose: Historical LTV calculations (compliance & disputes).

Column	Type	Description
id	BIGINT (PK)	Log ID
loan_id	UUID (FK → loans.id)	Loan reference
timestamp	TIMESTAMP	Evaluation time
btc_price_inr	DECIMAL	Median BTC price

calculated_ltv	DECIMAL	Resulting LTV
trigger_event	ENUM	ROUTINE_CHECK, MARGIN_CALL, LIQUIDATION

7. Negotiation & Audit Trail

Table: **negotiation_messages**

Purpose: Immutable chat history for legal evidence.

Column	Type	Description
id	BIGINT (PK)	Message ID
loan_id	UUID (FK → loans.id)	Loan reference
sender_id	UUID (FK → users.id)	Sender
message_text	TEXT	Message
is_system_message	BOOLEAN	System-generated flag
sent_at	TIMESTAMP	Sent time

8. Key Relationship Summary

- Users ↔ Loans (Borrower & Lender roles)
- Loan ↔ Escrow Account (1:1)
- Loan ↔ Platform Fee (1:1)
- Loan ↔ Bitcoin Transactions (1:N)
- Loan ↔ Repayments (1:N)

- Loan ↔ LTV Logs (1:N)
- Loan ↔ Negotiation Messages (1:N)

API Specification

LinkBit - API Specification

Base Configuration

- **Base URL:** `https://api.linkbit.app/v1`
 - **Auth:** `Authorization: Bearer <JWT>`
 - **Content-Type:** `application/json`
 - **Rule:** All endpoints enforce **strict loan-state validation**
-

1. Authentication & Identity

`POST /auth/login`

Purpose: Authenticate user and issue JWT.

Request

```
{
  "email": "user@example.com",
  "password": "plaintext_password"
}
```

Response

```
{
  "token": "eyJh...",
  "expires_in": 3600,
  "user_id": "uuid",
  "kyc_status": "VERIFIED"
}
```

```
}
```

POST /auth/kyc/submit

Purpose: Submit identity & bank details.

Security Rule

- `national_id` is **hashed immediately**
- Plaintext is **never stored or logged**

Request

```
{
  "full_name": "Akshay Kumar",
  "national_id": "ABCDE1234F",
  "bank_details": {
    "account_no": "...",
    "ifsc": "...",
    "upi_id": "..."
  }
}
```

Response

```
{
  "status": "PENDING_VERIFICATION"
}
```

2. Marketplace (Loan Offers)

GET /offers

Purpose: Browse anonymized loan offers.

Query Params

?amount=50000&tenure=90

Response

```
[
  {
    "offer_id": "uuid",
    "lender_pseudonym": "Satoshi_Lender",
    "max_loan_amount": 500000,
    "interest_range": "12-15%",
    "max_ltv": 65
  }
]
```

POST /offers

Purpose: (Lender only) Create a new loan offer.

Preconditions

- `user_type` $\in$ {LENDER, BOTH}
- `kyc_status` = VERIFIED

Request

```
{
  "max_loan_amount": 100000,
  "min_rate": 12,
  "max_rate": 18,
  "max_ltv": 65,
  "tenure_days": 90
}
```

```
}
```

Response

```
{  
  "offer_id": "uuid",  
  "status": "OPEN"  
}
```

3. Negotiation & Contract

POST /loans/connect

Purpose: (Borrower) Initiate negotiation.

Preconditions

- Offer status = OPEN
- Borrower KYC = VERIFIED

Request

```
{  
  "offer_id": "uuid",  
  "requested_amount": 50000  
}
```

Response

```
{  
  "loan_id": "uuid",  
  "status": "NEGOTIATING",  
  "chat_channel_id": "ch_123"
```

```
}
```

GET /loans/{id}/chat

Purpose: Fetch negotiation chat history.

Response

```
[
  {
    "sender": "LENDER",
    "text": "Can you do 14%",
    "timestamp": "2026-02-02T10:15:00Z"
  }
]
```

PUT /loans/{id}/terms

Purpose: (Lender only) Update live term sheet.

Preconditions

- Loan status = **NEGOTIATING**

Request

```
{
  "final_amount": 50000,
  "final_interest": 13.5,
  "duration_days": 90
}
```

POST /loans/{id}/finalize

Purpose: Freeze terms and generate contract hash.

System Actions

- Locks chat & terms
- Computes margin + liquidation prices
- Generates agreement hash

Response

```
{  
  "status": "AWAITING_SIGNATURES",  
  "contract_hash": "sha256_hash"  
}
```

POST /loans/{id}/sign

Purpose: Sign contract hash.

Rules

- Signer inferred from JWT
- Each party may sign **once**
- Duplicate signatures rejected

Request

```
{  
  "signature": "crypto_signature_string"  
}
```

Response

```
{  
  "status": "AWAITING_FEE"  
}
```

4. Platform Fee & Setup

POST /payments/fee/initiate

Purpose: Initiate processing fee payment.

Preconditions

- Loan status = `AWAITING_FEE`

Request

```
{  
  "loan_id": "uuid",  
  "gateway": "RAZORPAY"  
}
```

Response

```
{  
  "order_id": "order_123",  
  "amount_inr": 500,  
  "upi_intent_link": "upi://..."  
}
```

POST /webhooks/payment

Purpose: Payment gateway callback.

Rules

- Signature verification mandatory
- Idempotent handling

Action

- Update `platform_fees`
 - Loan status → `AWAITING_COLLATERAL`
-

5. Escrow & Collateral (Bitcoin)

`POST /loans/{id}/escrow/generate`

Purpose: Generate 2-of-3 multisig escrow address.

Preconditions

- Fee paid
- Loan status = `AWAITING_COLLATERAL`

Request

```
{  
  "borrower_public_key": "pubkey_hex"  
}
```

Notes

- Lender pubkey retrieved internally
- Platform key resolved from KMS

Response

```
{
  "address": "bc1q...",
  "redeem_script": "hex_string",
  "required_amount_btc": 0.015
}
```

POST /webhooks/bitcoin/deposit

Purpose: Bitcoin watcher callback.

Payload

```
{
  "txid": "...",
  "amount_sats": 1500000,
  "confirmations": 1
}
```

Rules

- Ignore duplicate TXIDs
- Confirmation threshold configurable

Action

- Loan status → **COLLATERAL_LOCKED**
-

6. Fiat Disbursement

GET /loans/{id}/payment-details

Purpose: (Lender) View borrower bank details.

Precondition

- Loan status = `COLLATERAL_LOCKED`
-

`POST /loans/{id}/disburse`

Purpose: (Lender) Mark fiat as sent.

Request

```
{  
  "transaction_ref_id": "IMPS123456",  
  "proof_image_url": "s3://..."  
}
```

System Actions

- Set `fiat_disbursed_at`
 - Trigger bank API verification
-

`POST /loans/{id}/confirm-receipt`

Purpose: (Borrower) Confirm receipt manually.

Rules

- If API verification succeeds and borrower is silent for **X hours**, loan auto-activates

Action

- Loan status → **ACTIVE**
 - Set **fiat_received_confirmed_at**
-

7. Active Loan Management

POST /loans/{id}/repay

Purpose: Submit EMI payment.

Request

```
{
  "amount": 5000,
  "transaction_ref": "UPI123",
  "proof_url": "s3://..."
}
```

Action

- System maps to due EMI
 - Status → **PAID_UNVERIFIED**
-

GET /loans/{id}/health

Purpose: Get real-time loan health.

Response

```
{
  "current_ltv": 68.5,
  "btc_price_inr": 5500000,
}
```

```
"status": "HEALTHY",  
"liquidation_price": 4800000  
}
```

8. Closure & Liquidation

POST /loans/{id}/release

Purpose: (Lender) Trigger collateral release.

Preconditions

- All EMIs paid
- Platform auto-signs internally

Response

```
{  
  "txid": "broadcasted_tx_hash"  
}
```

POST /loans/{id}/liquidate

Purpose: (Lender) Execute liquidation.

Preconditions

- Loan status = `LIQUIDATION_ELIGIBLE`

Response

```
{  
  "txid": "broadcasted_liquidation_tx",  
}
```

```
"lender_btc": 0.0123,  
"borrower_btc": 0.0021  
}
```

Loan State Machine Specification

LinkBit – Loan State Machine Specification

Design Principles

- **Single Source of Truth** for loan behavior
 - **State transitions are irreversible unless explicitly stated**
 - **Every transition has:**
 - Actor
 - Trigger
 - Preconditions
 - Side effects
 - **Platform enforces rules, not discretion**
-

1. Loan States (Canonical List)

NEGOTIATING

AWAITING_SIGNATURES

AWAITING_FEE

AWAITING_COLLATERAL

COLLATERAL_LOCKED

ACTIVE

MARGIN_CALL

LIQUIDATION_ELIGIBLE

SETTLEMENT_PENDING

CLOSED

LIQUIDATED

DISPUTE_OPEN

2. State Machine – Core Transition Table

Phase 1: Negotiation & Contract

Current State	Action	Actor	Preconditions	Side Effects	Next State
—	connect	Borrower	Offer = OPEN, Borrower KYC = VERIFIED	Create loan, open chat	NEGOTIATING
NEGOTIATING	update_terms	Lender	Loan = NEGOTIATING	Update live term sheet	NEGOTIATING
NEGOTIATING	finalize	Lender	Terms agreed	Freeze chat & terms, compute prices, generate agreement hash	AWAITING_SIGNATURES
AWAITING_SIGNATURES	sign	Borrower	Not already signed	Store borrower signature	AWAITING_SIGNATURES
AWAITING_SIGNATURES	sign	Lender	Not already signed	Store lender signature	AWAITING_FEE (if both signed)

Phase 2: Platform Fee & Escrow Setup

Current State	Action	Actor	Preconditions	Side Effects	Next State
AWAITING_FEE	initiate_fee	Borrower	Fee not paid	Create payment gateway order	AWAITING_FEE
AWAITING_FEE	fee_success	Payment Webhook	Valid signature	Record platform fee	AWAITING_COLLATERAL

AWAITING_COLLATERAL	generate_escrow	Borrower	Fee paid	Create multisig address	AWAITING_COLLATERAL
AWAITING_COLLATERAL	btc_deposit_detected	BTC Watcher	Tx seen (0 conf)	Log TX	AWAITING_COLLATERAL
AWAITING_COLLATERAL	btc_confirmed	BTC Watcher	$\geq$ required confirmations	Lock collateral	COLLATERAL_LOCKED

Phase 3: Fiat Disbursement

Current State	Action	Actor	Preconditions	Side Effects	Next State
COLLATERAL_LOCKED	view_payment_details	Lender	—	Reveal borrower bank details	COLLATERAL_LOCKED
COLLATERAL_LOCKED	mark_disbursed	Lender	—	Set fiat_disbursed_at, start verification	COLLATERAL_LOCKED
COLLATERAL_LOCKED	auto_verify_success	Bank API	Credit confirmed	Set fiat_received_confirmed_at	ACTIVE
COLLATERAL_LOCKED	manual_confirm	Borrower	Auto-verify failed	Set fiat_received_confirmed_at	ACTIVE
COLLATERAL_LOCKED	disbursement_timeout	System	X hours elapsed, no confirmation	Refund collateral	CLOSED

Invariant: If fiat transfer cannot be objectively verified, collateral returns to borrower.

Phase 4: Active Loan Lifecycle

Current State	Action	Actor	Preconditions	Side Effects	Next State
ACTIVE	repay	Borrower	EMI due	Record repayment	ACTIVE
ACTIVE	emi_verified	System / Lender	Payment confirmed	Update principal	ACTIVE
ACTIVE	ltv_check	Oracle	LTV $\geq$ warning threshold	Notify borrower	MARGIN_CALL
MARGIN_CALL	borrower_action	Borrower	Add BTC / repay	Recompute LTV	ACTIVE
MARGIN_CALL	ltv_breach	System	LTV $\geq$ liquidation threshold	Start cooldown timer	LIQUIDATION_ELIGIBLE

Phase 5: Liquidation & Default

Current State	Action	Actor	Preconditions	Side Effects	Next State
ACTIVE	repayment_overdue	System	> grace period	Flag default	LIQUIDATION_ELIGIBLE
LIQUIDATION_ELIGIBLE	liquidate	Lender	Cool-off elapsed	Generate PSBT, calculate fair split	LIQUIDATED
LIQUIDATION_ELIGIBLE	borrower_cure	Borrower	Before execution	Lower LTV	ACTIVE

Invariant: Liquidation is **never instant**; borrower always gets a final window.

Phase 6: Normal Closure

Current State	Action	Actor	Preconditions	Side Effects	Next State
---------------	--------	-------	---------------	--------------	------------

ACTIVE	loan_repaid	System	Outstanding = 0	Generate release tx	SETTLEMENT_PENDING
SETTLEMENT_PENDING	sign_release	Lender	—	Platform auto-signs	CLOSED

■ Exceptional & Dispute States

Current State	Action	Actor	Preconditions	Side Effects	Next State
ANY	open_dispute	System	Contradictory claims	Freeze actions	DISPUTE_OPEN
DISPUTE_OPEN	resolve_refund	Admin + Borrower	Lender fault	Refund collateral	CLOSED
DISPUTE_OPEN	resolve_activate	Admin	Borrower fault	Activate loan	ACTIVE
DISPUTE_OPEN	resolve_liquidate	Admin + Lender	Clear default	Execute liquidation	LIQUIDATED

Invariant: Admin **cannot invent outcomes**, only enforce predefined ones.

3. Forbidden Transitions (Hard Rules)

These **must never happen**:

- ❌ NEGOTIATING → ACTIVE
- ❌ AWAITING_FEE → COLLATERAL_LOCKED
- ❌ ACTIVE → CLOSED (without repayment)
- ❌ LIQUIDATED → any other state
- ❌ CLOSED → any other state

4. Global Invariants

- 1. No fiat before BTC is locked**
- 2. No BTC movement with fewer than 2 signatures**
- 3. Collateral never moves without on-chain transaction**
- 4. Platform never signs unless protocol conditions are met**
- 5. Borrower always has a cure window before liquidation**
- 6. State transitions are atomic and idempotent**

Threat Model & Abuse Scenarios

LinkBit Threat Model & Abuse Scenarios (Authoritative)

Threat Modeling Scope

This model covers:

- Borrowers
- Lenders
- Platform (internal + admin)
- External attackers
- Infrastructure failures

Threats are categorized by **who attacks**, **what they try**, **impact**, and **how LinkBit defends**.

1. Borrower-Originated Threats

Threat	Attack Vector	Impact	Mitigation	Residual Risk
Borrower takes fiat and refuses to repay	Intentional default	Lender loss	BTC collateral locked + liquidation rules	Market volatility
Borrower fakes EMI payment proof	Screenshot upload	Delayed lender payment	API verification overrides screenshots	Temporary delay
Borrower delays margin call response	Inaction	Increased lender exposure	Automated LTV monitoring + cooldown	BTC price gap
Borrower attempts double-spend BTC	RBF / low-fee TX	Collateral illusion	Confirmation threshold + mempool monitoring	Minor reorg risk
Borrower submits wrong pubkey	Malicious key	Escrow unusable	Key validation + test signing	User error

2. Lender-Originated Threats

Threat	Attack Vector	Impact	Mitigation	Residual Risk
Lender claims fiat sent when not sent	Fake transaction ref	Borrower BTC locked	API verification + refund-by-default rule	Short verification delay
Lender withholds release after repayment	Refuses to sign	Borrower collateral locked	Platform + borrower can sign refund	Platform availability
Lender forces premature liquidation	Timing manipulation	Borrower equity loss	Cool-off window + protocol checks	Oracle lag
Lender refuses to broadcast signed tx	Passive griefing	Funds stuck	Platform allowed to broadcast	None
Lender colludes with admin	Insider attack	Theft	2-of-3 multisig + audit logs	Insider risk (low)

3. Platform / Insider Threats

Threat	Attack Vector	Impact	Mitigation	Residual Risk
Platform steals BTC	Private key misuse	Catastrophic	2-of-3 multisig, no unilateral power	Collusion risk
Admin abuses dispute resolution	Manual override	Trust erosion	Predefined outcomes only + logs	Governance risk
Platform censors borrower	Denial of service	Locked funds	Borrower + lender can still transact	Infra downtime
Platform modifies contract terms	DB tampering	Legal exposure	Hash + signatures immutable	Insider DB access
KMS key compromise	Infra breach	Escrow risk	HSM/KMS policies + no full control	Extreme breach

4. External Attacker Threats

Threat	Attack Vector	Impact	Mitigation	Residual Risk
API credential theft	Phishing	Account takeover	JWT expiry + 2FA	User negligence
Replay of webhooks	Reused payloads	False state change	Signature verification + idempotency	None
Bitcoin oracle manipulation	Exchange spoofing	Forced liquidation	Median price from multiple sources	Black swan
DDoS on API	Traffic flood	Service downtime	Rate limiting + WAF	Temporary outage
Database breach	SQL injection	Data leak	Parameterized queries + encryption	Insider risk

5. Protocol-Level & Economic Attacks

Threat	Attack Vector	Impact	Mitigation	Residual Risk
Flash crash liquidation	Sudden BTC drop	Borrower liquidation	Median oracle + cooldown	Extreme volatility
BTC price gap down	Weekend crash	Under-collateralization	Lender receives 100% collateral	Lender loss
Fee grieving	Borrower pays fee then abandons	Resource waste	Small fixed fee	Minor cost
Liquidity fragmentation	Single-loan offers	Low capital efficiency	Future offer pooling	MVP limitation

6. Fiat Rail & Integration Risks

Threat	Attack Vector	Impact	Mitigation	Residual Risk
Bank API downtime	External dependency	Verification delay	Manual fallback + timeout rules	Delay

UPI reversal disputes	Bank quirks	Dispute escalation	Transaction reference + logs	Rare
Payment gateway webhook delay	Async failure	State desync	Idempotent processing	Temporary
Screenshot forgery	User upload	False claims	Screenshots never decisive	Delay only

7. Blockchain-Specific Risks

Threat	Attack Vector	Impact	Mitigation	Residual Risk
Bitcoin reorg	Chain reorg	False confirmation	Configurable confirmations	Very rare
Mempool congestion	High fees	Delayed liquidation	Fee estimation + RBF	Time risk
Invalid PSBT signing	Malformed tx	Locked funds	Pre-broadcast validation	Dev error
Address reuse	Privacy leak	Deanononymization	Unique escrow per loan	Minor

8. Global Safety Invariants (Non-Negotiable)

These invariants **block entire classes of attacks**:

1. Platform **cannot move BTC alone**
2. Fiat disputes default to **borrower refund**
3. Liquidation is **never instant**
4. No state transition without **valid preconditions**
5. Every critical action is **auditable**
6. Screenshots are **never authoritative**
7. On-chain truth beats off-chain claims

9. Threat Model Summary (Executive View)

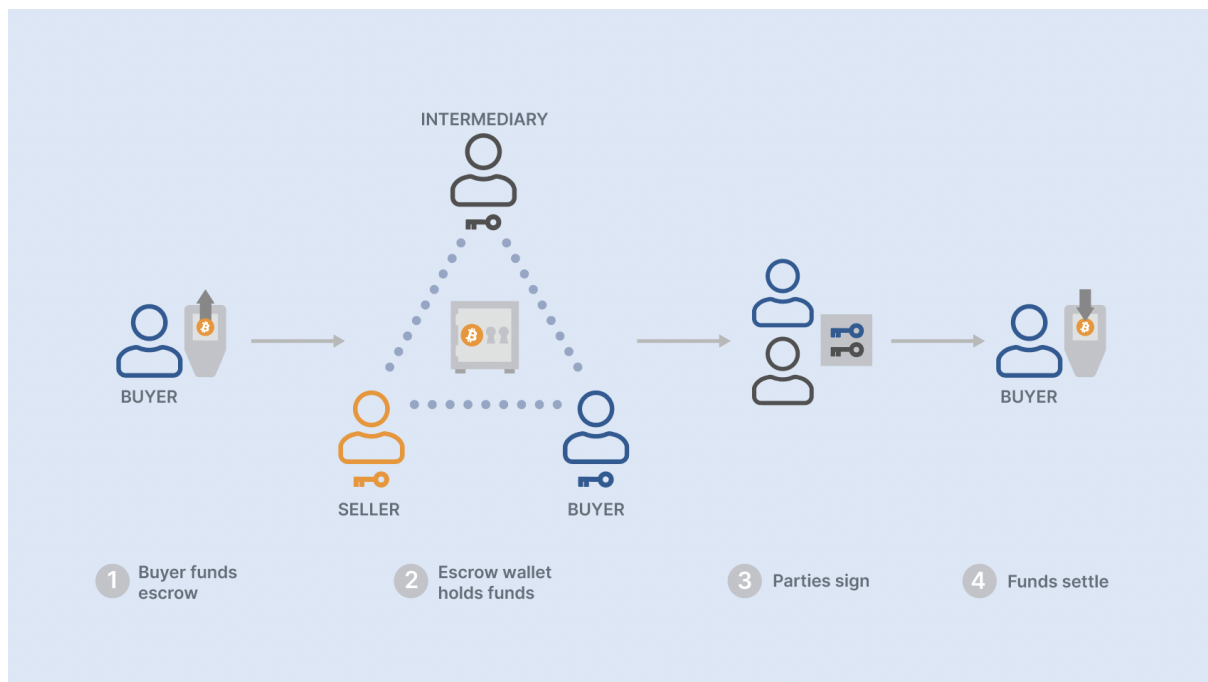
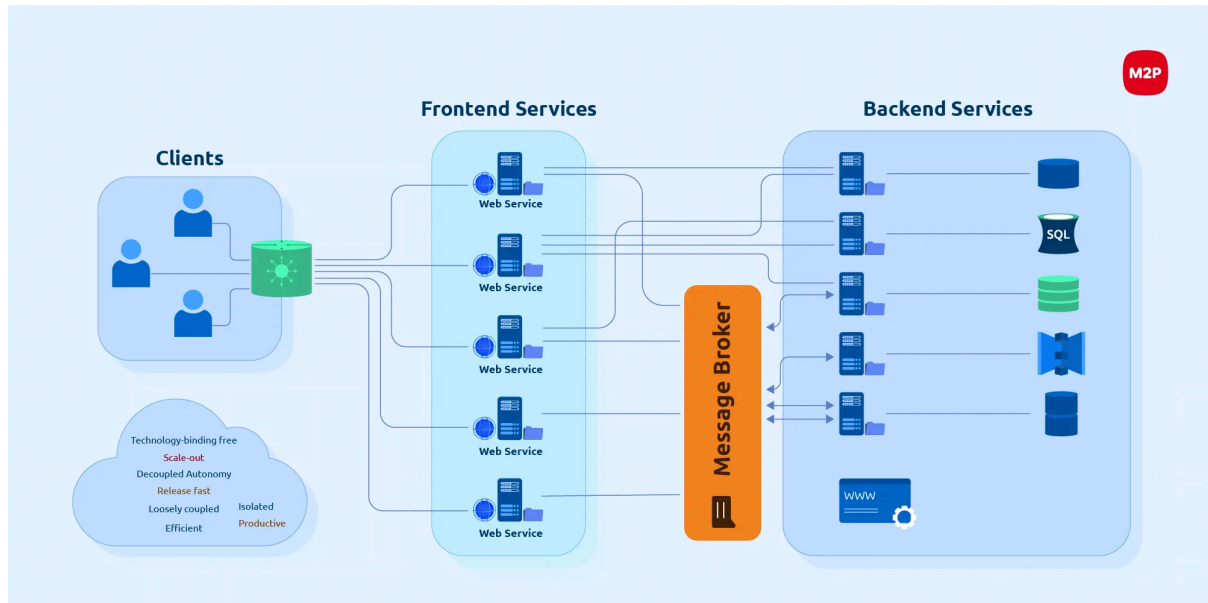
Category	Risk Level
Borrower abuse	Medium (collateralized)
Lender abuse	Medium (protocol-limited)
Platform abuse	Low (cryptographically constrained)
External hacking	Low–Medium
Market volatility	High (explicitly accepted risk)

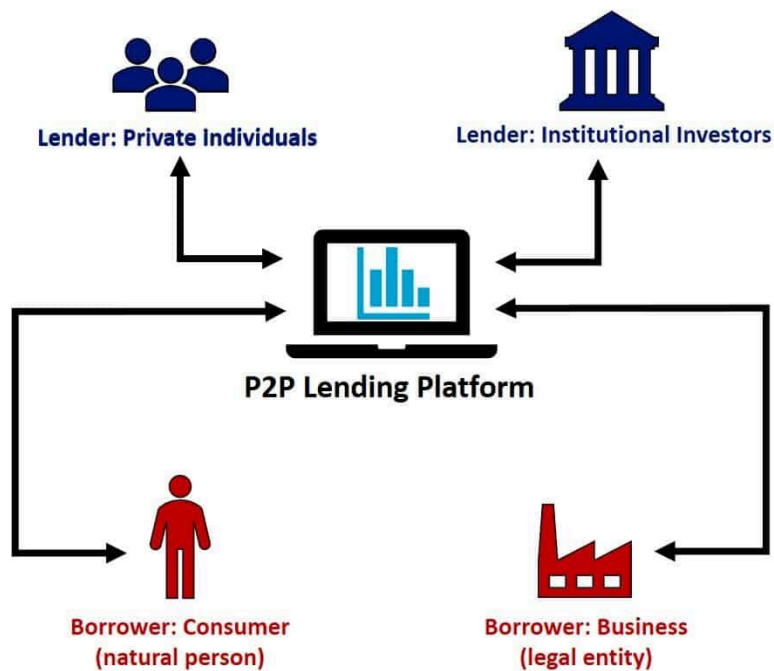
Key Insight:

LinkBit does not eliminate risk — it **compresses risk into explicit, visible, and enforceable rules**.

System Architecture

LinkBit – System Architecture





© P2Pmarketdata.com

1. Architectural Philosophy

LinkBit follows a “**Protocol-Enforced, Event-Driven Microservices**” model.

Key principles:

- **No single service is trusted**
- **State machine drives everything**
- **Critical paths are synchronous**
- **Risk & monitoring are asynchronous**
- **Crypto, fiat, and governance are isolated**

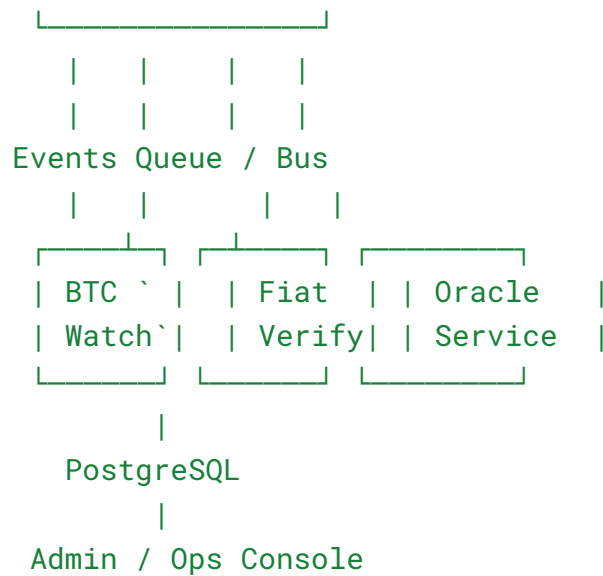
This is **not** a monolith and **not** an overkill mesh — it’s the *minimum safe architecture* for a crypto-fiat protocol.

2. High-Level System Overview

Clients (Web / Mobile)

|
API Gateway

|
┌ Core API Layer ─┐
└────────────────┘



3. Core Services (Authoritative)

3.1 API Gateway

Role: Entry point & policy enforcement

Responsibilities:

- JWT verification
- Rate limiting
- Request validation
- Route to internal services

Security:

- No business logic
 - Rejects invalid state transitions early
-

3.2 Core Loan Service (Brain of the System)

This is the MOST important service.

Responsibilities:

- Loan state machine enforcement
- Contract signing logic
- State transitions
- Invariant enforcement

Examples:

- Prevents **AWAITING_FEE** → **ACTIVE**
- Prevents liquidation without cooldown
- Guarantees atomic transitions

This service:

- Owns the **loans** table
 - Is the **only writer** of loan state
-

3.3 Identity & KYC Service

Responsibilities:

- Authentication
- JWT issuance
- KYC status tracking
- Sensitive data isolation

Design choice:

- KYC data **never touches** Core Loan Service directly
- Access via permissioned calls only

Compliance benefit:

- Blast-radius reduction
 - Easier audits
-

3.4 Escrow & Bitcoin Service

Responsibilities:

- Multisig address generation
- PSBT creation
- Signing coordination
- Transaction validation

Key rules:

- No private keys stored
- Platform key via KMS/HSM
- Stateless signing requests

Security boundary:

This service **cannot** change loan state directly
It emits events → Core Loan Service reacts

3.5 Bitcoin Watcher Service

Responsibilities:

- Monitor blockchain
- Detect deposits
- Track confirmations
- Publish immutable events

Design:

- Read-only
- Idempotent
- Stateless

Events emitted:

- `BTC_DEPOSIT_DETECTED`
 - `BTC_CONFIRMED`
-

3.6 Fiat Verification Service

Responsibilities:

- Payment gateway webhooks
- Bank / UPI verification
- Disbursement validation

Important:

- Screenshots treated as *signals*, not truth
- API verification overrides everything

Events:

- `FIAT_VERIFIED`
 - `FIAT_UNVERIFIED_TIMEOUT`
-

3.7 Oracle & Risk Engine

Responsibilities:

- Aggregate BTC prices
- Compute median price
- Calculate LTV
- Trigger margin calls

Design:

- Runs on schedule (cron / stream)
- Can **pause liquidations** if data unstable

Outputs:

- MARGIN_CALL_TRIGGERED
 - LIQUIDATION_ELIGIBLE
-

3.8 Notification Service

Responsibilities:

- Email
- SMS
- Push notifications

Triggered by:

- State changes
- Margin calls
- Deadlines
- Disputes

Important:

- Fully async
 - Never blocks state transitions
-

3.9 Admin / Ops Console

Responsibilities:

- View disputes
- Trigger predefined resolutions
- Audit logs

Critical restriction:

Admins **cannot invent states or transitions**

Every action:

- Logged
 - Replayable
 - Reviewable
-

4. Data Layer

Primary Database

- PostgreSQL
- Strong consistency
- Transactional guarantees

Ownership:

- Each service owns its tables
 - Core Loan Service is the source of truth
-

Event Bus / Queue

Used for:

- Bitcoin events
- Fiat verification
- Oracle updates
- Notifications

Why event-driven?

- Avoids tight coupling
 - Improves fault tolerance
 - Enables horizontal scaling
-

5. Security & Trust Boundaries

Explicit Trust Boundaries

Boundary

Why

API Gateway → Core Service	Prevents direct state mutation
Core Service → Escrow	Crypto isolation
Core Service → Fiat	No trust in fiat proofs
Platform Key	Never usable alone
Admin Console	Read-heavy, write-restricted

Key Security Guarantees

- Loan state is **append-only**
 - Every critical action is **signed or logged**
 - Multisig enforces crypto safety
 - No service can steal funds
 - No human can bypass protocol
-

6. Deployment Model

Recommended baseline:

- Containerized services
- Horizontal scaling for:
 - API Gateway
 - Core Loan Service
 - Watchers
- Background workers for:
 - Oracles
 - Notifications

Cloud-agnostic, but maps cleanly to **Amazon Web Services** / GCP / Azure.

Implementation Blueprint

LinkBit – Implementation Blueprint

0. Executive Summary (1 page version)

- **Core logic:** Java + Spring Boot (state machine, loans, APIs)
- **Bitcoin logic:** Rust (multisig, PSBT, watchers)
- **Async/events:** DB-backed Outbox Pattern (MVP-safe)
- **DB:** PostgreSQL + Flyway
- **Keys:** AWS KMS (platform key only)
- **Frontend:** Web-only (API-first)
- **Timeline:** 6–8 weeks
- **Team:** 4–6 engineers
- **Architecture style:** Modular monolith + isolated Rust services

Why this works:

You get **strong correctness** where it matters (Java), **crypto safety** where required (Rust), and **no infra overkill** for MVP.

1. Service Breakdown (Final)

1 Core API Service (Java / Spring Boot)

This is the brain of LinkBit

Responsibilities

- Loan state machine enforcement
- Contract lifecycle
- API endpoints

- Fiat disbursement logic
- Dispute handling
- Platform fee logic

Owns

- `users`, `loan_offers`, `loans`, `repayments`, `platform_fees`
- State transitions (single writer rule)

Key rules

- Only this service can mutate `loan.status`
 - All transitions go through a `StateTransitionService`
-

2 Bitcoin Service (Rust)

Crypto-critical, isolated

Responsibilities

- Multisig address generation
- Redeem script creation
- PSBT generation
- Transaction validation
- Platform signing via AWS KMS

Does NOT

- Change loan state
- Store private keys
- Talk directly to frontend

Interfaces

- gRPC or REST (internal only)
 - Called by Core API
 - Emits events via DB outbox
-

3 Bitcoin Watcher (Rust)

Read-only, stateless

Responsibilities

- Monitor blockchain via managed provider
- Detect deposits & confirmations
- Emit `BTC_DEPOSIT_DETECTED`, `BTC_CONFIRMED`

Rules

- Idempotent
 - No signing
 - No DB writes except event log
-

4 Oracle & Risk Worker (Java or Rust)

Asynchronous risk engine

Responsibilities

- Fetch BTC prices (multiple sources)
- Compute median price
- Calculate LTV
- Trigger margin calls / liquidation eligibility

Runs

- Scheduled (cron / worker)
 - Never blocks user flows
-

5 Notification Service (Optional for MVP)

Can be:

- In-process (initially)
 - Extracted later
-

2. Repository Structure (Recommended)

```
linkbit/
├── core-api/                (Java, Spring Boot)
│   ├── auth/
│   ├── loans/
│   ├── state-machine/
│   ├── disputes/
│   ├── payments/
│   ├── outbox/
│   └── config/
├── bitcoin-service/         (Rust)
│   ├── multisig/
│   ├── psbt/
│   ├── kms/
│   └── api/
├── bitcoin-watcher/        (Rust)
│   ├── watcher/
│   ├── providers/
│   └── events/
├── shared/
│   └── api-contracts/
```

```
|   └─ event-schemas/
|
└─ infra/
    ├── docker/
    ├── migrations/
    └─ env/
```

Keep **repos separate**.

Mixed-language mono-repos slow teams down.

3. Database & Migrations

Database

- PostgreSQL
- Strong transactions
- Serializable isolation for state transitions

Migrations

- Flyway
- Versioned SQL
- One-way migrations only

Golden Rule

State change = DB transaction + outbox insert

4. Async & Events (Your “Decide for me” → This)

Chosen Pattern: Transactional Outbox (MVP)

Why

- No Kafka/SQS complexity
- Exactly-once semantics achievable
- Perfect for 1–2 month MVP

How it works

1. Core API writes:
 - Loan state change
 - Outbox event (same transaction)
2. Worker polls outbox
3. Dispatches to:
 - Bitcoin service
 - Watcher reactions
 - Notifications

Upgrade path

- Swap dispatcher → Kafka/SQS later
- No business logic rewrite

5. State Machine Implementation (Critical)

Pattern

- **Explicit transition table in code**
- No `if/else` chains

Example (Java conceptually):

```
transition(  
  from = COLLATERAL_LOCKED,
```

```
    action = MARK_DISBURSED,  
    actor = LENDER,  
    to = ACTIVE,  
    preconditions = [...]  
)
```

Hard Rules

- One transition per transaction
 - Illegal transitions throw hard errors
 - All transitions logged
-

6. Bitcoin Integration Details

Managed Provider (MVP)

- Used for:
 - Watching deposits
 - Confirmation tracking
- Examples:
 - Blockstream
 - Alchemy
 - Mempool.space API

Signing

- Platform key:
 - AWS KMS
 - Restricted policy

- Borrower & lender sign client-side

PSBT Flow

1. Rust service builds PSBT
2. Platform signs via KMS
3. Lender signs
4. Broadcast

7. Security Controls (Non-Negotiable)

- JWT short-lived + refresh
- Role-based access
- Idempotency keys on:
 - payments
 - signing
 - liquidation
- Rate limiting on all write endpoints
- All webhooks signature-verified

8. Frontend Assumptions

- Web app only
- API-first
- No business logic in frontend
- All critical checks server-side

9. Team Allocation (4–6 Engineers)

Role	Responsibility
Backend Lead	Core API + state machine
Backend Dev	Payments, disputes, APIs
Rust Dev	Bitcoin service + watcher
Full-stack	Frontend + API integration
DevOps	Infra, CI/CD, secrets
QA / Support	Flows, edge cases

10. Build Timeline (Realistic)

Weeks 1–2

- Repo setup
- DB schema
- Auth + KYC
- Loan state machine skeleton

Weeks 3–4

- Offers + negotiation
- Escrow generation
- Bitcoin watcher
- Fee payments

Weeks 5–6

- Fiat disbursement

- LTV monitoring
- Liquidation flow
- Notifications

Weeks 7–8

- Hardening
- Edge cases
- Audit logs
- MVP freeze